

AEL-4

Graph Traversal - 2

Floyd-Warshall Algorithm

The Floyd Warshall Algorithm is an all pair shortest path algorithm unlike Dijkstra and Bellman Ford which are single source shortest path algorithms. This algorithm works for both the directed and undirected weighted graphs. But, it does not work for the graphs with negative cycles (where the sum of the edges in a cycle is negative). It follows Dynamic Programming approach to check every possible path going via every possible node in order to calculate shortest distance between every pair of nodes.

Obviously the shortest path between i to j will have some k number of intermediate nodes. The idea behind floyd warshall algorithm is to treat each and every vertex from 1 to N as an intermediate node one by one.

For $k = 0$ to $n - 1$

For $i = 0$ to $n - 1$

For $j = 0$ to $n - 1$

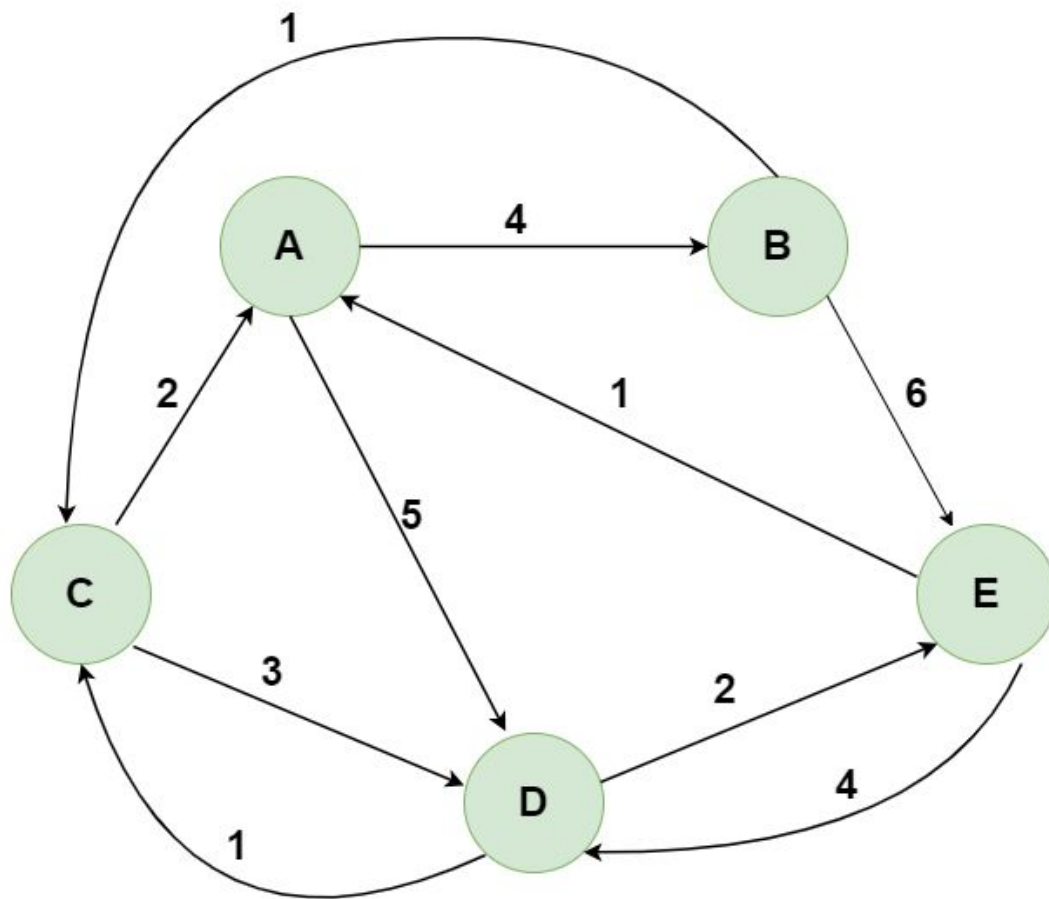
$Distance[i, j] = \min(Distance[i, j], Distance[i, k] + Distance[k, j])$

where i = source Node, j = Destination Node, k = Intermediate Node

Pseudo code

Illustration

Example Graph



Continued

Step1: Initializing Distance[][] using the Input Graph

	A	B	C	D	E
A	0	4	∞	5	∞
B	∞	0	1	∞	6
C	2	∞	0	3	∞
D	∞	∞	1	0	2
E	1	∞	∞	4	0

Step 2: Using Node A as the Intermediate node

$$\text{Distance}[i][j] = \min (\text{Distance}[i][j], \text{Distance}[i][A] + \text{Distance}[A][j])$$

	A	B	C	D	E
A	0	4	∞	5	∞
B	∞	?	?	?	?
C	2	?	?	?	?
D	∞	?	?	?	?
E	1	?	?	?	?



	A	B	C	D	E
A	0	4	∞	5	∞
B	∞	0	1	∞	6
C	2	6	0	3	12
D	∞	∞	1	0	2
E	1	5	∞	4	0

Step 3: Using Node B as the Intermediate node

$$\text{Distance}[i][j] = \min (\text{Distance}[i][j], \text{Distance}[i][B] + \text{Distance}[B][j])$$

	A	B	C	D	E
A	?	4	?	?	?
B	∞	0	1	∞	6
C	?	6	?	?	?
D	?	∞	?	?	?
E	?	5	?	?	?

	A	B	C	D	E
A	0	4	5	5	10
B	∞	0	1	∞	6
C	2	6	0	3	12
D	∞	∞	1	0	2
E	1	5	6	4	0

Step 4: Using Node C as the Intermediate node

$$\text{Distance}[i][j] = \min (\text{Distance}[i][j], \text{Distance}[i][C] + \text{Distance}[C][j])$$

	A	B	C	D	E
A	?	?	5	?	?
B	?	?	1	?	?
C	2	6	0	3	12
D	?	?	1	?	?
E	?	?	6	?	?

	A	B	C	D	E
A	0	4	5	5	10
B	3	0	1	4	6
C	2	6	0	3	12
D	3	7	1	0	2
E	1	5	6	4	0

Step 5: Using Node D as the Intermediate node

$$\text{Distance}[i][j] = \min (\text{Distance}[i][j], \text{Distance}[i][D] + \text{Distance}[D][j])$$

	A	B	C	D	E
A	?	?	?	5	?
B	?	?	?	4	?
C	?	?	?	3	?
D	3	7	1	0	2
E	?	?	?	4	?

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0

Step 6: Using Node E as the Intermediate node

$$\text{Distance}[i][j] = \min (\text{Distance}[i][j], \text{Distance}[i][E] + \text{Distance}[E][j])$$

	A	B	C	D	E
A	?	?	?	?	7
B	?	?	?	?	6
C	?	?	?	?	5
D	?	?	?	?	2
E	1	5	5	4	0

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0

Step 7: Return Distance[][] matrix as the result

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0

Important Facts about Floyd-Warshall

No matter how many edges are there in the graph the Floyd Warshall Algorithm runs for $O(V^3)$ times therefore it is best suited for Dense graphs. That's why Floyd-Warshall Algorithm better for Dense Graphs and not for Sparse Graphs.

- State some differences between Dijkstra's and floyd-warshall.
- State some difference between bellman-ford and floyd-warshall.

Minimum Spanning Tree

A **spanning tree** is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph. Or, to say in Layman's words, it is a subset of the edges of the graph that forms a tree (**acyclic**) where every node of the graph is a part of the tree.

The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees. Like a spanning tree, there can also be many possible MSTs for a graph.

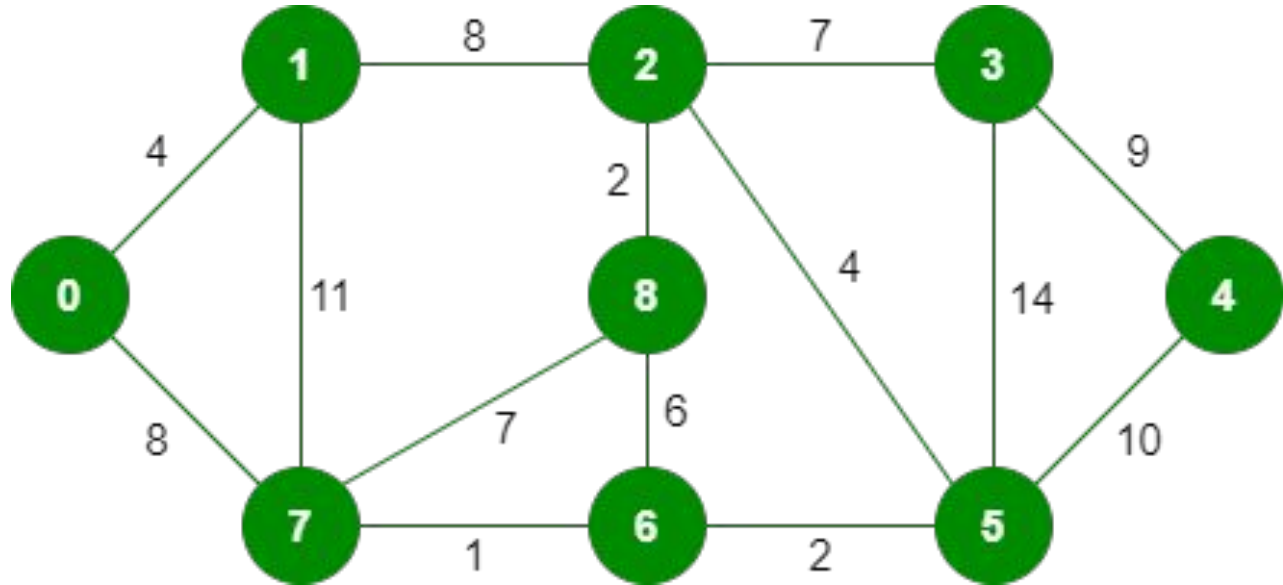
Properties of a Spanning Tree:

The spanning tree holds the **below-mentioned principles**:

- The number of vertices (**V**) in the graph and the spanning tree is the same.
- There is a fixed number of edges in the spanning tree which is equal to one less than the total number of vertices (**$E = V - 1$**).
- The spanning tree should not be **disconnected**, as in there should only be a single source of component, not more than that.
- The spanning tree should be **acyclic**, which means there would not be any cycle in the tree.
- The total cost (or weight) of the spanning tree is defined as the sum of the edge weights of all the edges of the spanning tree.
- There can be many possible spanning trees for a graph.

Kruskal MST Algorithm

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are $(V-1)$ edges in the spanning tree.



Union-Find Algorithm

Step 2 of the previous slide uses union-find algorithm to detect cycles. Has two operation Union and Find.

```
#include<bits/stdc++.h>
using namespace std;

int find(int i)
{
    // If i is the parent of itself
    if (parent[i] == i) {
        // Then i is the representative of
        // this set
        return i;
    }
    else {
        // Else if i is not the parent of
        // itself, then i is not the
        // representative of his set. So we
        // recursively call Find on its parent
        return find(parent[i]);
    }
}

// The code is contributed by Nidhi goel
```

Inefficient
approach could
take $O(n)$ in
worst case.

```
#include <bits/stdc++.h>
using namespace std;

void union(int i, int j) {
    // Find the representatives
    // (or the root nodes) for the set
    // that includes i
    int irep = this.Find(i),

    // And do the same for the set
    // that includes j
    int jrep = this.Find(j);

    // Make the parent of i's representative
    // be j's representative effectively
    // moving all of i's set into j's set
    this.Parent[irep] = jrep;
}
```

Almost Constant Approach

Path Compression (Used to improve find()):

The idea of path compression is to make the found root as parent of x so that we don't have to traverse all intermediate nodes again.

Union by Rank (Modifications to union()):

Rank is same as height if path compression is not used. With path compression, rank can be more than the actual height.

Now recall that in the Union operation, it doesn't matter which of the two trees is moved under the other. Now what we want to do is minimize the height of the resulting tree.

Cycle detection In Kruskal:

If parent of two node is same it means there is a cycle so we avoid that edge.

```
def find(self, i):
```

```
    root = self.parent[i]
```

```
    if self.parent[root] != root:
```

```
        self.parent[i] = self.find(root)
```

```
    return self.parent[i]
```

```
return root
```

```
def unionSets(self, x, y):
```

```
    xRoot = self.find(x)
```

```
    yRoot = self.find(y)
```

```
    if xRoot == yRoot:
```

```
        return
```

```
    # Union by Rank
```

```
    if self.rank[xRoot] < self.rank[yRoot]:
```

```
        self.parent[xRoot] = yRoot
```

```
    elif self.rank[yRoot] < self.rank[xRoot]:
```

```
        self.parent[yRoot] = xRoot
```

```
    else:
```

```
        self.parent[yRoot] = xRoot
```

```
        self.rank[xRoot] += 1
```

Prim's Algorithm

Step 1: Determine an arbitrary vertex as the starting vertex of the MST.

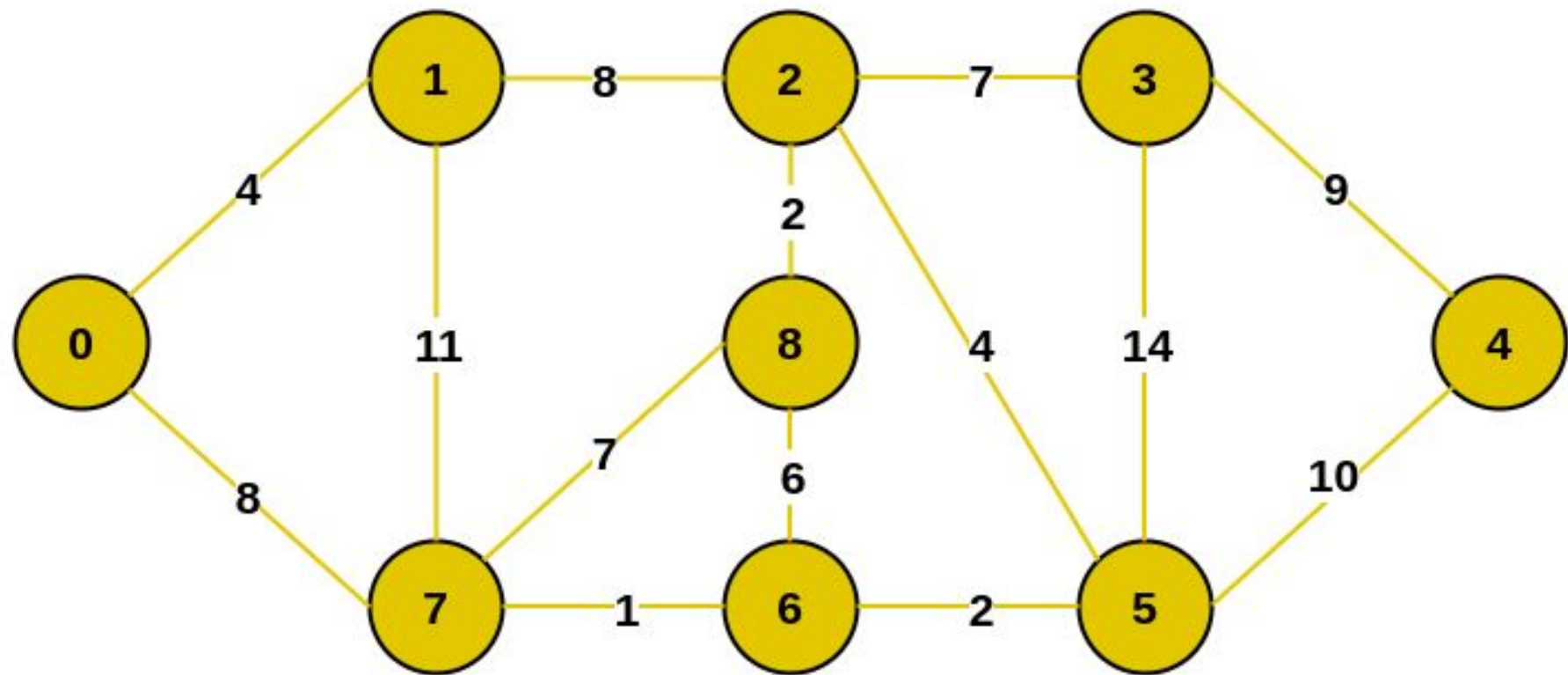
Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST if it does not form any cycle.

Step 6: Return the MST and exit



Example of a Graph

Network Design

A national internet service provider (ISP) plans to expand its fiber-optic network to connect several cities across Bangladesh. Connecting every pair of cities directly would be extremely expensive because laying fiber cables involves costs related to distance, terrain, labor, and infrastructure. The company's goal is to ensure that every city can communicate with every other city while minimizing the total installation cost. By representing each city as a node and the cost of connecting two cities as an edge weight, the company can construct a Minimum Spanning Tree (MST). The resulting MST identifies the set of cable connections that links all cities together at the lowest possible cost without creating redundant connections. In practice, algorithms such as Kruskal's algorithm are often suitable for this type of problem because the number of feasible cable routes is typically much smaller than the total number of possible city pairs, resulting in a sparse graph.

Clustering

An e-commerce company wants to improve its marketing strategy by grouping customers with similar purchasing behaviors. The company collects data such as annual spending, product preferences, shopping frequency, and average order value for thousands of customers. Each customer can be represented as a point in a multi-dimensional feature space, where the distance between two points indicates how similar the customers are. An MST can be constructed by treating customers as nodes and similarity distances as edge weights. After building the MST, the company can identify distinct customer groups by removing the longest edges, which typically connect otherwise separate clusters. For example, one cluster may represent budget-conscious shoppers, while another may consist of frequent premium buyers. Since customer clustering often involves comparing every customer with every other customer, the underlying graph is usually dense, making Prim's algorithm a practical choice for constructing the MST efficiently.

Image Segmentation

A medical imaging system is designed to assist radiologists in identifying tumors from MRI scans. In a digital image, each pixel can be represented as a node, and neighboring pixels are connected by edges whose weights reflect differences in intensity or color. **Pixels belonging to the same anatomical structure, such as healthy tissue, tend to have similar intensity values and therefore are connected by low-weight edges. In contrast, boundaries between different tissues or between a tumor and surrounding tissue produce higher-weight edges.** By constructing a Minimum Spanning Tree of the image graph and removing edges with unusually large weights, the system can separate the image into meaningful regions. This process helps isolate potential tumors from surrounding tissue, allowing radiologists to focus on suspicious areas more effectively. Because each pixel is connected only to a small number of neighboring pixels, the image graph is typically sparse, making Kruskal's algorithm combined with an efficient Union-Find data structure particularly well suited for large-scale image segmentation tasks.

Continued

The Right side is a 5x5 Gray scale image.

Now, we have the edge list, E where each element is a compound tuple of positions and weight. Like, ((sx,sy),(dx,dy),w). Weight is the absolute difference of intensity between two pixels. So the edge list E is,

E = [((0,0),(0,1),0), ((0,0),(1,0),0), ((0,0),(1,1),128), ((0,1),(0,0),0), ((0,1),(1,0),0), ((0,1),(1,1),128), ((0,1),(0,2),0), ((0,1),(1,2),192),]

Since, there are 25 pixels and almost 25x8 edges (a sparse graph), we perform kruskal. After performing kruskal we will have a MST. Now if we are considering a threshold and remove all the edges inside the MST that has higher value than that threshold we will divide the region in two or multiple parts.

Like, if we consider 60 as threshold and remove all the edges inside the MST that has higher weight than 60, we will end up with 6 parts. 3 parts of 0s, 1 part of 64s, 1 part of 128s, and 1 part of 255. Now if, different pixels means different things for instance, 0 means no tissue, 64 means normal tissue, 128 means benign tumor and 255 means malignant, then we can segment these using advanced thresholding techniques.

0	0	0	0	64
0	128	128	128	64
64	128	255	128	64
0	64	128	128	64
0	0	64	64	0